

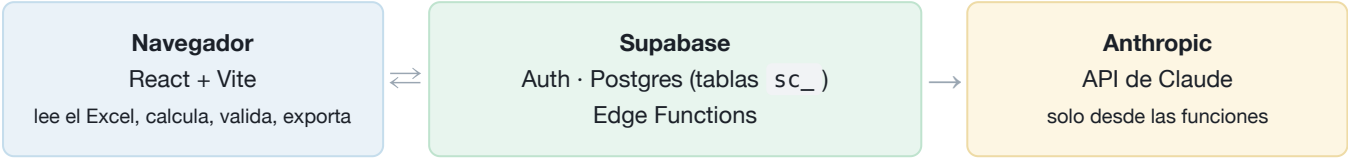
Despliegue e instalación

Cómo poner en marcha la app de Estados Financieros desde cero: repositorio, Supabase, Claude, base de datos, funciones y publicación.

-
- 1 Arquitectura
 - 2 Requisitos
 - 3 Repositorio
 - 4 Cuenta y proyecto en Supabase
 - 5 API key de Claude (Anthropic)
 - 6 Instalar el proyecto
 - 7 Variables de entorno
 - 8 Migrar la base de datos
 - 9 Edge Functions y secreto de Claude
 - 10 Usuarios y acceso
 - 11 Probar en local
 - 12 Publicar el frontend
 - 13 Operación y mantenimiento
 - 14 Seguridad y problemas conocidos
 - 15 Lista de verificación

1 Arquitectura

Una aplicación React que corre en el navegador, sobre Supabase (login, base de datos y dos funciones de servidor) y Claude para la clasificación asistida. El Excel se lee en el navegador: nunca se sube al servidor.



PIEZA	QUÉ HACE	DÓNDE VIVE
Frontend	Interfaz, lectura del Excel, clasificación de hojas, extracción, cálculos, validaciones, Excel y PDF	src/. Se compila a archivos estáticos (dist/) y se sirve con nginx.
Supabase Auth	Inicio de sesión con correo y contraseña	Proyecto de Supabase
Base de datos	Empresas, informes, cuentas, clasificaciones, memoria de correcciones, notas, validaciones, auditoría, catálogo institucional	Postgres del proyecto; esquema en supabase/migrations/
sc-identificar-hojas	Pide a Claude tipo y período de las hojas que las reglas no resolvieron	supabase/functions/
sc-clasificar-cuentas	Pide a Claude el rubro de las cuentas que las reglas no cubren	supabase/functions/
Claude	Modelo claude-sonnet-5 . Recibe texto (nombres de hojas y cuentas), nunca saldos	API de Anthropic, llamada solo desde las Edge Functions

POR QUÉ LA LLAVE DE CLAUDE NO ESTÁ EN EL FRONTEND

Todo lo que va en el frontend es visible para cualquiera que abra la app. Por eso la llave de Anthropic vive como secreto de Supabase y solo la leen las Edge Functions. El frontend solo conoce la URL del proyecto y la llave *publicable*, que está diseñada para ser pública y depende de las políticas de acceso (RLS) de la base.

2 Requisitos

Cuentas

CUENTA	PARA QUÉ	NOTAS
GitHub	Acceso al repositorio	Si el repositorio es privado, el propietario (<code>ivancclabe</code>) debe dar acceso a quien despliegue.
Supabase	Login, base de datos y funciones	El plan gratuito alcanza para empezar. Ver capítulo 4.
Anthropic	API de Claude	De prepago: requiere cargar créditos. Ver capítulo 5.

Herramientas en la máquina de quien despliega

HERRAMIENTA	VERSIÓN PROBADA	PARA QUÉ
Git	cualquiera reciente	Clonar el repositorio
Node.js y npm	Node 20 o superior	Instalar dependencias, correr y compilar. El contenedor de Docker usa Node 20.
Supabase CLI	2.114	Aplicar migraciones, desplegar funciones y cargar el secreto
Docker	29	Solo para publicar con el <code>Dockerfile</code> incluido

Instalación de la Supabase CLI:

```
# macOS
brew install supabase/tap/supabase

# Windows (con Scoop)
scoop bucket add supabase https://github.com/supabase/scoop-bucket.git
scoop install supabase

# Linux o cualquier sistema con Node: sin instalar, anteponiendo npx
npx supabase --version
```

3 Repositorio

GitHub	<code>https://github.com/ivancclabe/susconsultores-estados-financieros</code>
Rama principal	<code>main</code>
Copia de trabajo actual	<code>~/Workspace/susconsultores-estados-financieros</code> en el equipo de desarrollo

Estructura

RUTA	CONTENIDO
<code>src/components/</code>	Pantallas: ingreso, carga y clasificación de hojas, clasificación de cuentas, estados y aprobación
<code>src/lib/</code>	Lógica: lectura de Excel, clasificación de hojas, extracción, cálculos, validaciones, exportación, clientes de Supabase y de las funciones
<code>supabase/migrations/</code>	Los 4 scripts SQL del esquema, en orden
<code>supabase/functions/</code>	Las 2 Edge Functions que llaman a Claude
<code>scripts/</code>	Prueba del núcleo (197 comprobaciones) y diagnóstico de un Excel real
<code>docs/</code>	Manuales
<code>Dockerfile</code> , <code>nginx.conf</code> , <code>docker-compose.yml</code>	Publicación con Docker
<code>.env.example</code>	Plantilla de variables de entorno

4 Cuenta y proyecto en Supabase

Se recomienda un **proyecto dedicado** para SusConsultores. El proyecto de desarrollo actual está compartido con otras cuatro aplicaciones (ver capítulo 14).

Crear la cuenta

- 1 Entrar a `https://supabase.com` y pulsar **Start your project**.
- 2 Registrarse con GitHub o con correo. Si es con correo, confirmar el enlace que llega.
- 3 Crear una **organización** (por ejemplo, *SusConsultores*) y elegir el plan. El gratuito sirve para empezar; en él los proyectos se pausan tras una semana sin uso.

Crear el proyecto

- 1 En la organización, pulsar **New project**.
- 2 **Name:** por ejemplo `susconsultores-eeff`.
- 3 **Database password:** generar una contraseña fuerte y **guardarla** en un gestor de contraseñas. La pide la CLI al vincular el proyecto.
- 4 **Region:** la más cercana a los usuarios. Para Colombia, *South America (São Paulo)* o una región de Estados Unidos (este).
- 5 Pulsar **Create new project** y esperar uno o dos minutos a que termine de aprovisionarse.

Datos que se van a necesitar

DATO	DÓNDE ESTÁ	EJEMPLO
Project URL	Botón Connect del proyecto, o <i>Project Settings</i> → <i>Data API</i>	<code>https://abcdefghijkl.supabase.co</code>
Project ref	La parte de la URL antes de <code>.supabase.co</code>	<code>abcdefghijkl</code>
Publishable key	<i>Project Settings</i> → <i>API Keys</i>	<code>sb_publishable_...</code> (la llave <i>anon</i> clásica también funciona)
Database password	La que se definió al crear el proyecto	—

NUNCA EN EL FRONTEND

Las llaves *secret* o *service_role* saltan todas las reglas de acceso. No van en `.env.local` ni en el repositorio. Esta app no las necesita.

5 API key de Claude (Anthropic)

La API de Claude se paga por uso, con créditos cargados por adelantado. La llave se muestra una sola vez al crearla.

- 1 Entrar a `https://console.anthropic.com` y crear la cuenta (correo o Google). Confirmar el correo.
- 2 Completar los datos de la organización cuando la consola los pida.
- 3 Ir a **Settings** → **Billing** y cargar créditos. Sin saldo, la API responde con error.
- 4 Recomendado: en **Settings** → **Limits**, definir un límite de gasto mensual.
- 5 Ir a **Settings** → **API Keys** y pulsar **Create Key**. Nombrarla, por ejemplo, `susconsultores-produccion`.
- 6 **Copiar la llave en ese momento** (empieza por `sk-ant-`) y guardarla en un gestor de contraseñas. No se puede volver a ver; si se pierde, se crea otra.

Modelo que usa la app	<code>claude-sonnet-5</code> , en la constante <code>MODELO</code> de cada función (<code>supabase/functions/*/index.ts</code>)
Dónde se configura la llave	Solo como secreto de Supabase: <code>ANTHROPIC_API_KEY</code> (capítulo 9)
Qué recibe Claude	Nombres, títulos y encabezados de hojas; códigos y nombres de cuentas. Nunca saldos.
Cuándo se llama	Solo cuando las reglas no alcanzan. Con el archivo de prueba de 2025-2024 no hizo falta ninguna llamada.

DECISIÓN DE SUSCONSULTORES

Aunque no viajan saldos, sí salen de la oficina los nombres de cuentas y de hojas de los clientes (por ejemplo, el nombre de un banco o de un tercero). Confirmar con la firma que eso es aceptable antes de activar la IA. Sin la llave, la app funciona igual con reglas y asignación manual.

6 Instalar el proyecto

```
# 1. Clonar
git clone https://github.com/ivanclabe/susconsultores-estados-financieros.git
cd susconsultores-estados-financieros

# 2. Instalar dependencias (usa package-lock.json: versiones exactas)
npm ci

# 3. Comprobar que compila
npm run typecheck
```

La dependencia `xlsx` se descarga desde el CDN oficial de SheetJS (`cdn.sheetjs.com`), no desde npm. Si la red lo bloquea, `npm ci` falla en ese paso.

7 Variables de entorno

```
cp .env.example .env.local
```

Editar `.env.local` con los datos del capítulo 4:

```
VITE_SUPABASE_URL=https://abcdefghijkl.supabase.co
VITE_SUPABASE_ANON_KEY=sb_publishable_XXXXXXXXXXXXXXXXXXXX
```

VARIABLE	USO
VITE_SUPABASE_URL	URL del proyecto. La usa el login, la base y las llamadas a las funciones.
VITE_SUPABASE_ANON_KEY	Llave publicable. Pese al nombre, sirve tanto la <code>sb_publishable_...</code> como la <i>anon</i> clásica.

SE LEEN AL COMPILAR, NO AL EJECUTAR

Vite incrusta estas variables en los archivos compilados. Si cambian, hay que volver a compilar (`npm run build` o reconstruir la imagen de Docker). Sin ellas, la app muestra «Falta configuración». `.env.local` está en `.gitignore`: nunca se sube al repositorio.

8 Migrar la base de datos

Cuatro scripts en `supabase/migrations/` crean todo el esquema. Se aplican en orden y solo una vez.

ARCHIVO	QUÉ HACE
<code>20260922120000_sc_init.sql</code>	Tablas base (empresas, rubros, reglas, informes, cuentas, clasificaciones, memoria, validaciones, auditoría) y reglas de acceso (RLS)
<code>20260922120100_sc_catalogo_semilla.sql</code>	Catálogo provisional: 19 rubros y 34 reglas de mapeo por código PUC
<code>20260923090000_sc_extraccion_completa.sql</code>	Jerarquía completa, terceros, trazabilidad a celda y tablas de notas del archivo
<code>20260925090000_sc_notas_hoja.sql</code>	Hoja de origen de cada nota

Opción A — con la Supabase CLI (recomendada)

```
# Una sola vez: crea supabase/config.toml (no toca migraciones ni funciones)
supabase init

# Iniciar sesión en la CLI (abre el navegador)
supabase login

# Vincular con el proyecto. Pide la Database password del capítulo 4
supabase link --project-ref abcdefghijkl

# Aplicar las migraciones pendientes. Muestra la lista y pide confirmación
supabase db push
```

Opción B — desde el panel de Supabase

En **SQL Editor** → **New query**, pegar y ejecutar (**Run**) cada archivo, en el orden de la tabla. Si más adelante se usa la CLI, registrar las cuatro versiones con `supabase migration repair --status applied <versión>`.

Verificar

En el SQL Editor:

```
select (select count(*) from sc_rubros)      as rubros,    -- 19
       (select count(*) from sc_reglas_mapeo) as reglas,   -- 34
       (select count(*) from information_schema.tables
        where table_schema = 'public' and table_name like 'sc\_%') as tablas; -- 11
```

NO USAR DB PUSH CONTRA EL PROYECTO DE DESARROLLO ACTUAL

Es un proyecto compartido con otras apps y las cuatro migraciones **ya están aplicadas** ahí, con otras versiones: `db push` se detendría pidiendo reconciliar el historial. Los cambios nuevos en ese proyecto van por el SQL Editor.

9 Edge Functions y secreto de Claude

Opción A — con la CLI

```
# El secreto. Queda disponible de inmediato, sin volver a desplegar
supabase secrets set ANTHROPIC_API_KEY=sk-ant-xxxxxxxxxxxxxxxxx

# Las dos funciones (exigen una sesión válida: verify_jwt activo por defecto)
supabase functions deploy sc-identificar-hojas
supabase functions deploy sc-clasificar-cuentas

# Comprobar
supabase secrets list
supabase functions list
```

Opción B — desde el panel

- 1 **Edge Functions** → **Secrets**: agregar `ANTHROPIC_API_KEY` con la llave del capítulo 5 y guardar.
- 2 **Edge Functions** → **Deploy a new function** → **Via Editor**.
- 3 Nombre exacto `sc-identificar-hojas`. Reemplazar el código de ejemplo por el de `supabase/functions/sc-identificar-hojas/index.ts` y desplegar.
- 4 Repetir con `sc-clasificar-cuentas`.
- 5 Dejar activa la verificación de JWT en ambas.

Verificar

```
curl -X POST https://abcdefghijkl.supabase.co/functions/v1/sc-identificar-hojas \
-H "Authorization: Bearer sb_publishable_xxxx" -H "apikey: sb_publishable_xxxx" \
-H "content-type: application/json" -d '{"hojas":[]}'
```

RESPUESTA	SIGNIFICA
<code>{"error": "Payload inválido."}</code> (400)	Todo bien. La función está desplegada y el secreto existe; solo rechazó el cuerpo vacío de la prueba.
<code>...falta el secreto ANTHROPIC_API_KEY...</code> (500)	Falta el secreto: repetir <code>supabase secrets set</code> .
401 / <code>Invalid JWT</code>	La llave del encabezado no es de este proyecto.
404	La función no está desplegada o el nombre no coincide.

La prueba de punta a punta con Claude se hace desde la app: si en el paso 2 se clasifican cuentas con «Usar el modelo» marcado y ninguna regla las cubre, deben aparecer propuestas con la etiqueta *¡a*.

10 Usuarios y acceso

La app no tiene pantalla de registro: los usuarios los crea el administrador.

Crear un usuario

- 1 **Authentication** → **Users** → **Add user** → **Create new user**.
- 2 Correo y contraseña. Marcar **Auto Confirm User** para que pueda entrar de inmediato.
- 3 Entregar las credenciales por un canal seguro.

Cerrar el registro público — obligatorio

En **Authentication** → **Sign In / Providers**, desactivar **Allow new users to sign up**.

POR QUÉ ES OBLIGATORIO

Las reglas de acceso dan acceso completo a cualquier usuario con sesión. Con el registro abierto, cualquiera que conozca la URL y la llave publicable (ambas visibles en la app) podría crearse una cuenta por la API y leer la información financiera de los clientes.

Opcional

En **Authentication** → **URL Configuration**, poner como *Site URL* el dominio de la app. Lo usan los correos de recuperación de contraseña.

11 Probar en local

```
# Servidor de desarrollo en http://localhost:5199
npm run dev

# Prueba del núcleo: 197 comprobaciones sobre libros sintéticos (sin red ni base)
npx esbuild scripts/prueba-nucleo.ts --bundle --platform=node --format=cjs \
  --outfile=/tmp/prueba.cjs && node /tmp/prueba.cjs

# Diagnóstico de un Excel real: qué clasifica y extrae, sin escribir nada
npx esbuild scripts/diagnostico-archivo.ts --bundle --platform=node --format=cjs \
  --outfile=/tmp/diag.cjs && node /tmp/diag.cjs "ruta/al/archivo.xlsx"
```

Recorrido mínimo antes de publicar:

- 1 Entrar con un usuario creado en el capítulo 10.
- 2 Cargar un Excel real. La clasificación de hojas debe quedar completa, sin revisión pendiente.
- 3 Revisar los tres tabs y las validaciones. Continuar.
- 4 Clasificar las cuentas, confirmar y ver los estados. *Activo = Pasivo + Patrimonio* debe cuadrar.
- 5 Aprobar y descargar el Excel.

12 Publicar el frontend

El repositorio trae un `Dockerfile` de dos etapas: compila con Node 20 y sirve los archivos con nginx. El `docker-compose.yml` está pensado para correr detrás de un proxy inverso que da el dominio y el HTTPS.

En el servidor (Linux con Docker)

```
# 1. Traer el código
git clone https://github.com/ivanclabe/susconsultores-estados-financieros.git
cd susconsultores-estados-financieros

# 2. Variables: se incrustan al compilar la imagen
cp .env.example .env.local
nano .env.local          # VITE_SUPABASE_URL y VITE_SUPABASE_ANON_KEY

# 3. La red "web" del proxy inverso (solo si no existe)
docker network create web

# 4. Compilar y levantar
docker compose up -d --build
```

PIEZA	COMPORTAMIENTO
<code>Dockerfile</code>	<code>npm ci</code> y <code>npm run build</code> en <code>node:20-alpine</code> ; copia <code>dist/</code> a <code>nginx:alpine</code> .
<code>nginx.conf</code>	Puerto 80, gzip, caché larga para <code>/assets/</code> , <code>index.html</code> sin caché (cada despliegue se ve de inmediato) y respaldo a <code>index.html</code> .
<code>docker-compose.yml</code>	Un servicio en la red externa <code>web</code> , sin puertos publicados : el proxy inverso (Traefik, Caddy, nginx-proxy...) lo alcanza por esa red y se encarga del dominio y el certificado.

SIN PROXY INVERSO

Para exponer el contenedor directamente, agregar al servicio `ports: ["8080:80"]` y quitar la red externa. Así queda sin HTTPS: sirve para pruebas internas, no para producción.

Actualizar a una versión nueva

```
git pull
docker compose up -d --build
```

Si la versión trae migraciones o cambios en las funciones, aplicarlos antes (capítulos 8 y 9).

Alternativa: hosting estático

En Netlify, Vercel, Cloudflare Pages o similar: comando de build `npm run build`, carpeta de salida `dist`, y las dos variables `VITE_...` configuradas en el panel del servicio. La app no usa rutas, así que no necesita reglas de reescritura.

13 Operación y mantenimiento

TAREA	CÓMO
Ver errores de la IA	Panel de Supabase → Edge Functions → la función → Logs .
Consumo de Claude	Consola de Anthropic → Usage . Solo hay llamadas cuando las reglas no alcanzan.
Rotar la llave de Claude	Crear una nueva en la consola, <code>supabase secrets set ANTHROPIC_API_KEY=...</code> y borrar la anterior. No requiere volver a desplegar.
Cambiar de modelo	Editar la constante <code>MODELO</code> de las dos funciones y volver a desplegarlas.
Cambio de esquema	<code>supabase migration new nombre</code> , escribir el SQL, <code>supabase db push</code> . Nunca editar una migración ya aplicada.
Respaldos	Los planes pagos de Supabase hacen copias diarias. En el gratuito: <code>supabase db dump -f respaldo.sql</code> periódicamente.
Usuarios	Authentication → Users : crear, restablecer contraseña o eliminar.

Reemplazar el catálogo institucional

El catálogo que viene con la app es **provisional**: está armado sobre el PUC colombiano. El de SusConsultores se carga en las tablas `sc_rubros` (rubros, sección, número de nota, orden) y `sc_reglas_mapeo` (prefijo PUC → rubro). La app lo lee al iniciar sesión: no hay que tocar código.

```
-- Ejemplo: renombrar un rubro y moverlo de nota
update sc_rubros
  set nombre = 'Efectivo y equivalentes de efectivo', nota_numero = 4, origen = 'susconsultores'
 where codigo = 'EFECTIVO';

-- Ejemplo: una regla nueva, más específica (gana el prefijo más largo)
insert into sc_reglas_mapeo (prefijo, rubro_codigo, prioridad, origen)
values ('1330', 'OTROS_ACT_CTE', 10, 'susconsultores');
```

Las secciones válidas del Estado de Situación Financiera son `activo_corriente`, `activo_no_corriente`, `pasivo_corriente`, `pasivo_no_corriente` y `patrimonio`. Las del Estado de Resultados son `ingresos`, `costos`, `gastos`, `otros_ingresos`, `otros_gastos` e `impuesto`.

14 Seguridad y problemas conocidos

Lo que conviene resolver antes de usar la app con información real de clientes.

TEMA	SITUACIÓN	QUÉ HACER
Proyecto compartido	El proyecto de desarrollo también aloja otras cuatro apps con sus propios usuarios. Como las reglas dan acceso a cualquier usuario con sesión, los usuarios de esas apps podrían leer los datos de SusConsultores .	Producción en un proyecto dedicado (capítulo 4).
Registro público	Si queda activo, cualquiera puede crearse una cuenta y leer todo.	Desactivarlo (capítulo 10).
Funciones de IA sin sesión	Las funciones aceptan la llave publicable aunque no haya sesión, así que alguien con la app podría llamarlas y gastar créditos.	Límite de gasto en Anthropic (capítulo 5). Mejora pendiente: que la función verifique que hay un usuario con sesión.
Sin <code>.dockerignore</code>	<code>COPY . .</code> copia también <code>node_modules</code> y <code>dist</code> locales. Compilando en un Mac con dependencias instaladas, los binarios de macOS pisan los de Linux y el build puede fallar.	Compilar desde un clon limpio en el servidor, o agregar un <code>.dockerignore</code> con <code>node_modules</code> y <code>dist</code> . No excluir <code>.env.local</code> : de ahí salen las variables del build.
Nombres heredados	El servicio de <code>docker-compose.yml</code> se llama <code>dark-kitchen</code> , y los comentarios de <code>nginx.conf</code> mencionan rutas de otra app.	Renombrar el servicio (por ejemplo, <code>sus-consultores</code>) y ajustar la regla del proxy. Los comentarios no afectan el funcionamiento.
Informes de prueba	En la base de desarrollo quedan tres informes creados con una versión anterior de la extracción, con datos incorrectos.	Descartarlos. No existen en un proyecto nuevo.
Plan gratuito	Supabase pausa el proyecto tras una semana sin actividad y no hace respaldos.	Plan pago para producción, o respaldos manuales.

15 Lista de verificación

- ☐ Acceso al repositorio de GitHub.
- ☐ Proyecto de Supabase **dedicado**, con la contraseña de la base guardada.
- ☐ Cuenta de Anthropic con créditos, límite de gasto y llave guardada.
- ☐ `npm ci` y `npm run typecheck` sin errores.
- ☐ `.env.local` con la URL y la llave publicable.
- ☐ Las cuatro migraciones aplicadas; la verificación da 19 rubros, 34 reglas y 11 tablas.
- ☐ Secreto `ANTHROPIC_API_KEY` cargado y las dos funciones desplegadas; la prueba con `curl` responde «Payload inválido».
- ☐ Registro público **desactivado** y usuarios creados a mano.
- ☐ Prueba del núcleo: 197 comprobaciones correctas.
- ☐ Recorrido completo con un Excel real: carga, clasificación de hojas, clasificación de cuentas, estados, aprobación, descarga.
- ☐ Frontend publicado con HTTPS.
- ☐ Catálogo institucional de SusConsultores cargado (o confirmado el provisional).
- ☐ SusConsultores autorizó enviar nombres de cuentas a Claude.